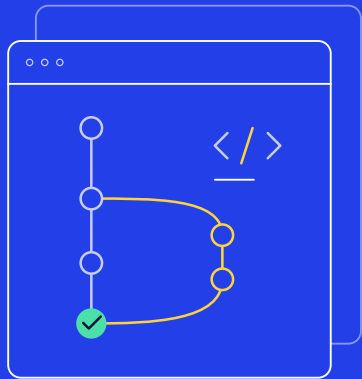


Shared Repository, Git Workflow & Code Quality Checks

Start your team's Bike Demand workflow.



PART 1

Set up the repo and team workflow

Agree how the team will work.



01

Get access

02

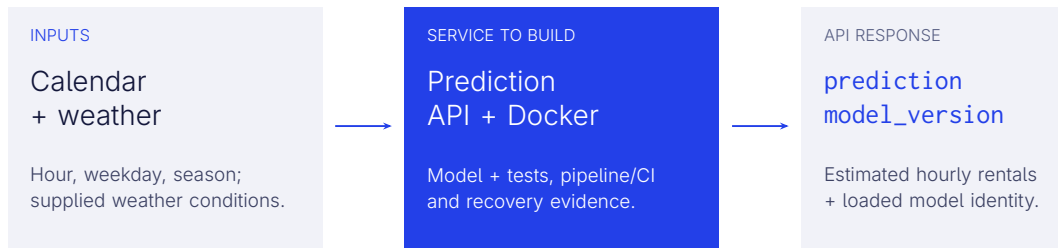
Read the starter

03

Agree the workflow

One **Bike Demand project**, built across the course

Estimate total hourly rentals from the supplied calendar and weather inputs.



Lab 01 starts the **shared workflow**. Lab 02 introduces data and training.

Eight labs build the same service

Weeks 1–4 develop the system; Week 5 verifies and presents it.

Week 1	Lab 1 / Repository, peer review, checks Lab 2 / Data, split, baseline, tracking	m1
Week 2	Lab 3 / Dependency lock, Docker, clean rerun Lab 4 / Prediction API, validation, tests	m2
Week 3	Lab 5 / Deployment, logs, model identity Lab 6 / GitHub Actions, regression tests	m3
Week 4	Lab 7 / Training pipeline, quality gate Lab 8 / Incident diagnosis, verified rollback	m4

Week 5 / Clinic, final verification and presentations; no new lab or report.

m1–m4 are ungraded checkpoints: link check results and remaining blockers. Dates are published separately.

Final handover: one service, with evidence

Week 5: GitHub Release v1.0.0 + team presentation (60%).

Release v1.0.0

Run

Code/tests, trained model,
locked setup + Docker instructions.

Verify

Link revision, configuration and model
to metrics, test/CI results and recovery.

Hand over

README, recovery runbook
and traceable release notes.

Submission dates are published separately.

Presentation flow

Explain

User + scope; data split, model
and evaluation against the baseline.

Demonstrate

Valid API call + loaded model version;
invalid-input behavior.

Support with evidence

Tests/CI, gate and recovery evidence;
each member explains their contribution.

Two assessment components; no separate lab grade

The team project and each student's own reflection contribute different evidence.

60%

Team project + presentation

One integrated system, verification evidence and the final team presentation.

$4 \times 10\%$

Individual reports

One private report after each of Weeks 1–4; 40% in total.
Due Sunday 23:59 of the same week.



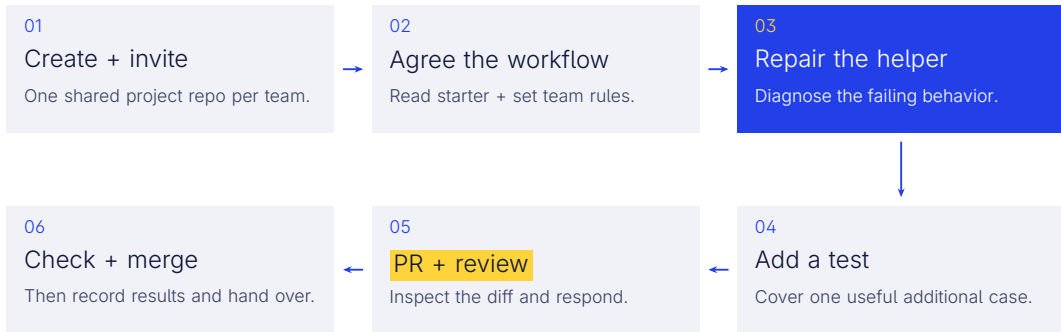
Team component

Four equally weighted reports

Team checkpoints m1–m4 support feedback; they carry no marks.

Today: one small change, from access to handover

The team owns the change, review and merge. We do not train a model today.



One team of four; different authors and reviewers

Work in a team of 4 (3 if needed). No fixed subteams or inter-group handover.

EXAMPLE ROTATION, LAB 1 TO LAB 8

	Week 1		Week 2		Week 3		Week 4	
	1	2	3	4	5	6	7	8
A	L			R	L			R
B	R	L			R	L		
C		R	L			R	L	
D			R	L			R	L

■ L = lead author ■ R = reviewer

Team of 3: rotate A, B, C. The reviewer is never the lead author.

1 Verify access and setup

One shared repo; one private report repo per student. Invite instructor, register links and run preflight.

2 Name a lead and a reviewer

Reviewer is another member; no self-approval. Every member writes code or a substantive review.

3 Rotate the lead every lab

A different member leads each lab PR; over 8 labs, each of 4 leads twice. Record leads and reviewers in notes.

Name repositories so staff can find them

Use the same names for the whole course; do not rename or recreate them.

TEAM REPOSITORY

mlops-thu1-sparrows-bike-demand

course

session

team name

project

One per team, created by one member.
Every member and the instructor have access.
Holds code, tests, PRs and team worksheet.

Example: mlops-thu1-sparrows-bike-demand

PERSONAL REPOSITORY

mlops-thu1-<github-username>

course

session

your GitHub username

One per student; keep it private.
Invite instructor minhtc-uca only; no teammates.
Holds your reports and their screenshots.

Example: mlops-thu1-minhtc

Session: thu1, thu2, fri1, fri2 (1 = morning, 2 = afternoon). Team name: lowercase letters and hyphens.

Read the starter before changing it

Separate what the approved package supplies from what your team must complete.

 your-team / bike-demand Private

src/	Starter helper
tests/	Behavior tests
configs/	Project settings
README.md	Setup and tasks
WORKING-AGREEMENT.md	Team rules
pyproject.toml	Project metadata
uv.lock	Locked dependencies

[README.md](#)

Illustrative layout, not a real repository.

Find these parts

src/: the helper

tests/: the behavior contract

Configuration + team worksheet

Your contribution

The helper repair, **one useful test**,
a reviewed PR and actual evidence.

Keep code, tests, PRs and handover
in the same team repository.

Before coding: agree how your team will work

Write three short agreements in the shared repository; use them in your first PR.

01

Project scope

In README.md:
user + goal;
inputs + output;
scope + non-goals;
how success is checked.

02

Contribution rules

In CONTRIBUTING.md:
branch + commit naming;
local checks;
PR + peer review;
who merges and when.

03

Team plan

In brief working notes:
availability + help route;
next steps + blockers;
handover: who did what.

Done when a teammate can pick up a task and know how to get it reviewed and merged.

Choose your conventions; keep the course scope, peer review and checks.

PART 2

From a shared repo to the first change

Complete one small task that a teammate
can test, inspect and review.

Small scope. Checkable outcome.



Repair the helper; defend your test

One supplied behavior test fails. Diagnose it from pytest, not a worked answer.

1

Meet the existing behavior contract

`normalize_team_slug` trims, joins whitespace with hyphens, produces lowercase labels and rejects blank names with `ValueError`.

2

Choose one useful additional test

Keep the change small. Explain what your case protects that the supplied tests do not; do not weaken assertions, skip tests or disable checks.

3

Deliver a reviewed, checked PR

Include code and test, another member's review and your response. Recheck this revision before merge; record actual results and blockers.

Separate setup faults from exercise faults

Use the approved starter from its project root. These are setup/check commands, not results.

APPROVED STARTER ONLY

One behavior failure is intentional. Read its assertion yourself. Sync/import errors or extra failures need diagnosis.

SETUP + DIAGNOSIS

```
uv sync --locked
uv run --locked python -m bike_demand.preflight
uv run --locked pytest -q -m infra
uv run --locked pytest -q -m lab1
```

Missing lock or blocked setup? Ask for support; do not replace the approved environment.

Make a small, reviewable Git change

Branch, edit, inspect and check before you commit and push.

EXAMPLE BRANCH: CHOOSE A NAME ...

```
git status --short  
git switch -c lab-01-workflow
```

01 Edit the helper + add a test

02 Inspect the diff

03 Run the local checks

EXAMPLE COMMANDS: USE TEAM CONVENTIONS ...

```
git diff  
git add src/bike_demand/team.py tests/lab1/test_team.py  
git commit -m "Normalize team labels"  
git push -u origin lab-01-workflow
```

Stage only intended files; use the approved access route.
Never expose credentials in commands, commits or screenshots.

Branch → Code + test → Diff → Checks → Commit → Push

Run local checks, read the result, then recheck

Testing, linting and formatting answer different questions. A command is not a result.

01 RUFF Code quality

Is the code consistent?

02 INFRA Setup health

Is the supplied setup healthy?

03 LAB 1 Behavior

Does the label follow its contract?

TERMINAL / READ EACH CHECK'S OWN SIGNAL

```
uv run --locked ruff check .  
uv run --locked ruff format --check .  
uv run --locked pytest -q -m infra  
uv run --locked pytest -q -m lab1
```

Then run all tests

AFTER THE FIX

```
uv run --locked pytest -q
```

No secrets, environments or
generated artifacts in Git.

Record **actual outcomes**; expectations are not measurements.

Review the reasoning, not just green checks

Another member in your group reviews the actual code and test before approval.

1

Inspect the fix against the contract

Does the diff explain the failing behavior without breaking valid inputs or the existing rejection behavior?

2

Challenge the added test

What useful case is new? Which plausible defect would it catch? Assertions should verify behavior, not mirror implementation.

3

Respond, recheck and merge

Record a finding and the author's response. Confirm checks on the current revision; then approve and merge, or name the blocker.

Remote checks belong to a specific revision

Local success does not show that GitHub ran the same checks.

 Normalize team labels #1 [Open](#)

lab-01-workflow into main

Checks for commit [a1b2c3d](#)

✓	ruff check	Successful in 9s	Details
✓	pytest -m infra	Successful in 14s	Details
✗	pytest -m lab1	Failing after 12s	Details

Each run is tied to one revision; open Details for the job log.

Illustrative layout, not a real pull request.

Before merging

- 01 Match the check to the revision you intend to merge.
- 02 Read the actual outcome; open logs when a check fails.
- 03 Confirm review + response and record any unresolved blocker.

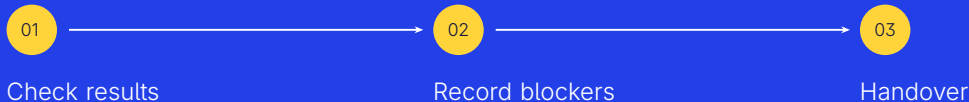
Class rule: GitHub does not enforce the merge automatically.

Remote run blocked? Record why; do not claim a pass.

PART 3

From a repaired change to completion evidence

Show what actually happened.



One team record; each student explains their work

Link to shared evidence. Explain your own contribution; do not copy the team worksheet.

TEAM / LAB 1 WORKSHEET

reports/lab-01.md

EVIDENCE

Code/test PR + checked revision;
each member's commit or useful review.

RECORD

Test choice + reason + actual result;
Review/check links; owners + blockers.

ASSESSMENT

Shared, ungraded worksheet.
m1 follows review of both labs.

INDIVIDUAL / PRIVATE WEEK 1 REPORT

reports/week-1.md

EVIDENCE

Your contribution + evidence;
one decision or debugging example.

RECORD

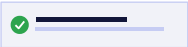
Private repo you create;
invite the instructor; submit on main.

ASSESSMENT

10%; due Sunday 23:59, same week.
No report PR or CI; keep it private.

Example: the team worksheet

Kept in the team repository. Values are illustrative: record what actually happened.

reports/lab-01.md		EXAMPLE
Team and roles	mlps-thu1-sparrows-bike-demand; A leads, B reviews	
Setup check	Preflight run by every member; outcome recorded per person	
PR and revision	PR #1 into main; checked commit a1b2c3d	
Added test	<the case you chose>; protects <the defect it would catch>	
Failures	lab1 failed before the fix; no infrastructure failures	
CI result	ruff, infra and lab1 on a1b2c3d: passed	
Review	B asked for one more case; C added it and replied	
Lab 2 handover	Next lead B, reviewer C; blockers: none	
Screenshots	 lab1: fail, then pass	 PR #1 checks, a1b2c3d

1 Links, not claims
A PR and checked commit
let anyone verify the work.

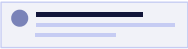
2 Name each failure
Separate the intentional
failure from setup faults.

3 Show who did what
Roles and handover make
ownership visible.

Each line is a claim someone can verify from a link, a commit or a captioned screenshot.

Example: your individual Week 1 report

Kept in your private repository. Write it in your own words from your own work.

reports/week-1.md		EXAMPLE
Student and team	<your name>, team mlops-thu1-sparrows-bike-demand	
Code and PR links	My commit a1b2c3d on PR #1; my review comment on PR #1	
Execution results	uv run --locked pytest -q: passed on a1b2c3d, my terminal	
My contribution	I wrote the added test and reviewed the helper repair	
Decisions	I chose <the case> because <the defect it would catch>	
Review and response	Reviewer asked for one more case; I added it (link)	
Blockers	CI run delayed; recorded, then rechecked on the same commit	
Assistance	Starter code; teammate B explained the failing assertion	
Screenshots	 pytest on a1b2c3d	 review comment, PR #1

- 1 Your own words
Do not copy the team worksheet; write what you did.
- 2 Show your evidence
Link your commits and PR, and add captioned screenshots.
- 3 State the gaps
Record blockers, missing evidence and any help received.

Finish Lab 1; rotate into Lab 2

Keep the same project repository. The Week 1 report covers both labs.

1

Lab 1 completion

Working setup, workflow agreement, code + test PR, independent peer approval, current passing checks and merged commit.

2

A short evidence check, not a full demo

Record test choice, reason and result with PR/check links. Any member may be asked: what does your added test protect?

3

Lab 2 lead must be a different member

Use the approved data/training extension. Keep reports/week-1.md private (10% of the grade). m1 follows both labs; no separate lab grade.

Deadline: Sunday 23:59 of the same week (published time zone).